RV32IM CPU in Verilog

Design Notes: Stage 1 and Stage 2

Jonathan Avalos

October 10, 2026

Contents

1 Overview **3**

2 Shared encodings **5**

3 Arithmetic and logic **5**
3.1 addersub . . . 5
3.2 logics . . . 6
3.3 shifter . . . 6
3.4 compare . . . 7
3.5 alu . . . 7
3.6 multdiv . . . 7

4 Control **8**
4.1 immgen . . . 8
4.2 decoder . . . 9
4.3 branch_unit . . . 9

5 State **10**
5.1 regfile . . . 10
5.2 reg32 . . . 10
5.3 memory . . . 11

6 Top level **12**
6.1 core . . . 12
6.2 top . . . 13

7 Verification **13**
7.1 Approach . . . 13
7.2 Results . . . 14
7.3 Test program . . . 14
7.4 Building and running . . . 14

8 Test program walkthrough **15**
8.1 How the program reports results . . . 15

8.2 Overview 15
8.3 Setup 16
8.4 Math (boxes 0–3) 16
8.5 Big numbers (boxes 4–6) 16
8.6 Logic (boxes 7–11) 17
8.7 Shifts (boxes 12–17) 17
8.8 Compares (boxes 18–21) 18
8.9 Loads and stores (boxes 22–27) 19
8.10 Branches (box 28) 20
8.11 Loop (box 29) 20
8.12 Function calls (boxes 30–32) 21
8.13 Multiply and divide (boxes 33–43) 22
8.14 Fibonacci (box 44) 23
8.15 x0 (box 45) 23
8.16 Done (box 46) 23
8.17 Final register values 24

9 Known limitations **24**

10 Next steps **24**

1 Overview

This document covers the CPU as it stands at the end of Stage 2: a single-cycle RV32IM core (the base integer instructions plus multiply and divide) written in Verilog. The long-term goal is to run Linux on a Nexys A7-100T board.

Status

- **Stage 1:** single-cycle RV32I core. Done.
- **Stage 2:** M extension (`mul`, `div`, `rem` and variants). Done. Passes the `rv32ui` and `rv32um` riscv-tests except `ma_data` (misaligned access).
- Every module has its own self-checking testbench, and a 141-instruction test program checks the whole CPU (Section 7).
- **Next:** a 5-stage pipeline, then Stage 3 (CSRs, traps, timer, MMIO).

Module hierarchy

```
top
  core
    reg32           32 registers (31 x regfile, x0 hardwired)
    immgen          immediate generator
    decoder         control unit
    alu
      logics        xor / or / and
      compare       signed and unsigned less-than
      shifter       sll / srl / sra
      addersub      add / sub
    branch_unit
      compare
    multdiv         M extension
  memory            one memory for instructions and data
```

Datapath

Every instruction finishes in one clock cycle. The PC goes out to memory, the instruction comes back, and everything else happens combinationally. On the rising clock edge the register write, the memory write and the PC update all happen together.

The full datapath is in Figure 1 on the next page.

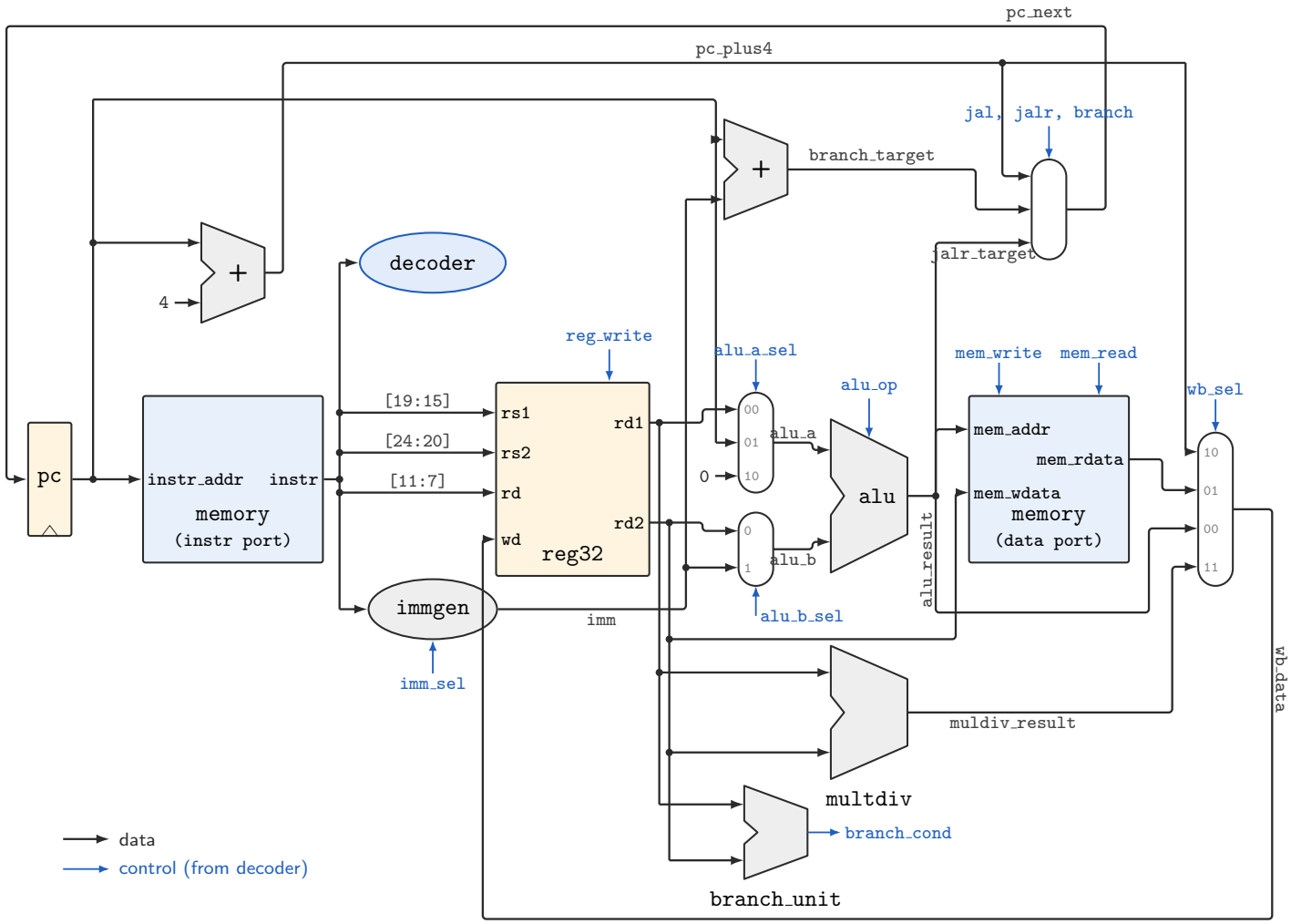


Figure 1: Single-cycle datapath. Every name matches the Verilog: blocks are module names, labels on blocks are port names, labels on wires are the wire names in `core.v`, and blue labels are `decoder` outputs. The small numbers on each mux input are the select code that picks it. `reg32`'s `rd1`/`rd2` drive `rs1_data`/`rs2_data`. The PC mux is the `pc_next` logic: `jalr_target` on `jalr`, `branch_target` on `jal` or (`branch` and `branch_cond`), otherwise `pc_plus4`.

2 Shared encodings

These codes are produced by the decoder and used by the other modules. They have to agree everywhere, so they are collected here.

`alu_op`: {`instr[30]`, `funct3`}

`alu_op`	op	`alu_op`	op
`0000`	add	`0100`	xor
`1000`	sub	`0101`	srl
`0001`	sll	`1101`	sra
`0010`	slt	`0110`	or
`0011`	sltu	`0111`	and

The code is just bit 30 of the instruction glued onto `funct3`, so for R-type instructions the decoder copies it straight out of the instruction.

Mux selects

signal	value	meaning
`imm_sel`	`000 / 001 / 010 / 011 / 100`	I / S / B / U / J format
`alu_a_sel`	`00 / 01 / 10`	rs1 / PC / zero
`alu_b_sel`	`0 / 1`	rs2 / immediate
`wb_sel`	`00 / 01 / 10 / 11`	ALU / memory / PC+4 / multdiv

3 Arithmetic and logic

3.1 addersub

File: `addersub.v` **Purpose:** 32-bit add or subtract.

port	width	
`a`, `b`	32 in	operands
`sub`	1 in	0 = add, 1 = subtract
`sum`	32 out	result
`cout`	1 out	carry out of bit 31

```
assign b_xor = b ^ {32{sub}};
assign {cout, sum} = a + b_xor + sub;
```

Subtraction is done as $a + \neg b + 1$, which is $a - b$ in two's complement. Line by line:

- `{32{sub}}` is Verilog's *replication* operator: 32 copies of the 1-bit `sub`. It's all zeros when adding and all ones when subtracting.
- `b ^ {32{sub}}`: XOR with all zeros leaves `b` unchanged, XOR with all ones flips every bit, giving $\neg b$. So `b_xor` is `b` for add and $\neg b$ for subtract.

- {cout, sum} = ...: a *concatenation* on the left side makes one 33-bit target, cout on top and sum below. The addition is done at 33 bits, so the carry out of bit 31 lands in cout instead of being dropped.
- + sub adds the extra 1 that two's complement needs. For an add it adds 0, so the result is just $a + b$.

Notes

- After a subtract, cout = 1 means *no* borrow, and cout = 0 means $a < b$ unsigned.
- RISC-V has no flags register, so cout isn't used by the CPU. Synthesis removes it.
- Signed overflow is not detected. RISC-V add just wraps.

Testbench: addersub_tb.v, 8 checks. Simple add and subtract, a negative result, unsigned wraparound, signed overflow and underflow, $0 - 0$ and $0 - 1$ (the two cout cases).

3.2 logics

File: logics.v **Purpose:** bitwise xor, or, and.

op	result
00	a ^ b
10	a | b
11	a & b
01	0 (unused)

The op codes are the low two bits of alu_op (funct3 of 100, 110, 111), so the ALU passes alu_op[1:0] in without any decoding.

Testbench: logics_tb.v, 6 checks. All three ops, the unused code, x ^ x = 0, and xor with all ones as "not".

3.3 shifter

File: shifter.v **Purpose:** shift by 0–31.

shift_op	op	fills with
00	sll	zeros from the right
01	srl	zeros from the left
11	sra	copies of the sign bit
10	0 (unused)	

shift_op is alu_op[3:2]: sll is 0001, srl is 0101, sra is 1101. The shift amount is only b[4:0], as the ISA requires. sra uses $signed(a) >>> shamt.

Testbench: shifter_tb.v, 11 checks. Shift by 0 and 31 for each type, sra on negative and positive values, and the unused code.

3.4 compare

File: `compare.v` **Purpose:** less-than, signed and unsigned.

```
assign lt  = $signed(a) < $signed(b);
assign ltu = (a < b);
```

- `$signed(a) < $signed(b)`: Verilog compares vectors as *unsigned* by default. `$signed()` tells it to read the bits as two's complement, so `FFFFFFFF` counts as -1 instead of $2^{32} - 1$. Both sides need it: if either operand is unsigned, Verilog quietly does the whole comparison unsigned.
- `(a < b)`: the plain, unsigned comparison.
- A comparison gives a 1-bit true/false, so `lt` and `ltu` are single bits.

Used twice: inside the ALU (for `slt`, `sltu`) and inside the branch unit (for `blt`, `bge`, `bltu`, `bgeu`).

Testbench: `compare_tb.v`, 7 checks. Includes `80000000` vs `7FFFFFFF` and `FFFFFFFF` vs `0`, where signed and unsigned give opposite answers.

3.5 alu

File: `alu.v` **Purpose:** pick the right result for `alu_op`.

All four sub-units (`logics`, `compare`, `shifter`, `addersub`) run on every cycle. A `case` on `alu_op` picks which result comes out. Unused codes give 0. `alu_op[3]` doubles as the adder's `sub` input, which is why add is `0000` and sub is `1000`.

Testbench: `alu_tb.v`, 14 checks. Every op, `sll` by 33 (only the low 5 bits count), `sltu` against zero, `srl` and `sra` on the same input, and an unused code.

3.6 multdiv

File: `multdiv.v` **Purpose:** the M extension.

`funct3`	instr	result
`000`	mul	low 32 bits of $a \times b$
`001`	mulh	high 32, signed $\times$ signed
`010`	mulhsu	high 32, signed $\times$ unsigned
`011`	mulhu	high 32, unsigned $\times$ unsigned
`100`	div	signed quotient, rounds toward zero
`101`	divu	unsigned quotient
`110`	rem	signed remainder, sign of a
`111`	remu	unsigned remainder

The multiplies extend both inputs to 64 bits by hand and then do a plain 64-bit multiply:

```
wire [63:0] a_s = {{32{a[31]}}, a};   // a sign-extended
wire [63:0] a_u = {32'b0, a};          // a zero-extended
wire [63:0] b_s = {{32{b[31]}}, b};   // b sign-extended
wire [63:0] b_u = {32'b0, b};          // b zero-extended
```

```
wire [63:0] prod_ss = a_s * b_s;       // mulh
wire [63:0] prod_su = a_s * b_u;       // mulhsu
wire [63:0] prod_uu = a_u * b_u;       // mulhu

wire div_by_zero = (b == 32'b0);
wire overflow    = (a == 32'h8000_0000) && (b == 32'hFFFF_FFFF);
```

- {{32{a[31]}}, a} is *sign extension*: 32 copies of the sign bit a[31], glued on top of a. A negative number gets 32 ones on top, a positive one gets 32 zeros, so the 64-bit value means the same number.
- {32'b0, a} is *zero extension*: always 32 zeros on top, so a is read as unsigned.
- Once both inputs are the right 64-bit values, an ordinary 64-bit multiply gives the right 64-bit product (two's complement multiplication works the same as unsigned when you only keep 64 bits). So the three products differ only in how the inputs were extended, which is exactly the difference between **mulh**, **mulhsu** and **mulhu**.
- mul takes prod_uu[31:0]: the low 32 bits of a product are the same no matter how the inputs were extended, so any of the three would do.
- div_by_zero and overflow flag the two special cases below, so the case can return the RISC-V-defined answers before ever doing the divide.

Special cases (RISC-V never traps on divide):

case	div / divu	rem / remu
divide by zero	all ones (FFFFFFFF)	a
$-2^{31}/-1$ (signed only)	-2^{31}	0

Testbench: multdiv_tb.v, 21 checks. Every instruction, negative operands, both special cases, and the sign of rem.

4 Control

4.1 immgen

File: immgen.v **Purpose:** pull the immediate out of the instruction and sign-extend it.

format	imm_sel	bits (high to low)
I	000	instr[31:20]
S	001	instr[31:25], instr[11:7]
B	010	instr[31], instr[7], instr[30:25], instr[11:8], 0
U	011	instr[31:12], 12'b0
J	100	instr[31], instr[19:12], instr[20], instr[30:21], 0

All formats except U fill the upper bits with copies of instr[31]. The sign bit is always instr[31], in every format, so the hardware can start sign-extending before it knows the format.

Testbench: `immgen_tb.v`, 18 checks, using real encoded instructions. Each format has a positive and a negative case, plus cases that set *one* immediate bit at a time (e.g. `beq +2048`). The first version of this test missed a swapped bit in the B format; the one-bit cases were added to catch that.

4.2 decoder

File: `decoder.v` **Purpose:** turn the opcode into control signals.

Every output starts at 0, then the `case` on the opcode sets only what that instruction needs. Anything not listed (unknown opcodes, `fence`, `ecall`) leaves everything at 0, so it does nothing.

instr	`reg_write`	`imm_sel`	A	B	`alu_op`	mem	`wb_sel`	jump/branch
R-type	1	–	rs1	rs2	{i30,f3}	–	ALU	–
M ext.	1	–	rs1	rs2	–	–	mul	–
OP-IMM	1	I	rs1	imm	see below	–	ALU	–
load	1	I	rs1	imm	add	read	mem	–
store	0	S	rs1	imm	add	write	–	–
branch	0	B	–	–	–	–	–	`branch`
lui	1	U	zero	imm	add	–	ALU	–
auipc	1	U	PC	imm	add	–	ALU	–
jal	1	J	–	–	–	–	PC+4	`jal`
jalr	1	I	rs1	imm	add	–	PC+4	`jalr`

Details

- **OP-IMM `alu_op`:** only shifts (`funct3 = 101`) use `instr[30]`. For everything else bit 30 is part of the immediate, so it's forced to 0. Without this, `addi x1, x0, -1` (immediate `FFF`, bit 30 set) would decode as subtract.
- **M extension:** detected by `instr[25] = 1` on the R-type opcode. The ALU result is ignored and `wb_sel` picks multdiv.
- **Branches** don't use the ALU. The branch unit compares rs1 and rs2, and the core adds PC + imm for the target.

Testbench: `decoder_tb.v`, 31 checks. All 18 control bits are packed into one vector, so each instruction is one comparison. Covers every RV32IM instruction type, `addi -1` and `xori -1` (the bit-30 trap), `srai` vs `srli`, and four things that must do nothing: `fence`, `ecall`, all zeros, all ones.

4.3 branch_unit

File: `branch_unit.v` **Purpose:** decide if a branch is taken.

`funct3`	instr	taken when
000	beq	$a = b$
001	bne	$a \neq b$
100	blt	$a < b$
101	bge	$a \geq b$
110	bltu	$a < b$ (unsigned)
111	bgeu	$a \geq b$ (unsigned)

"Greater or equal" is just "not less than", so `bge` and `bgeu` reuse the same comparator. `funct3` `010` and `011` are not branches and give 0.

Testbench: `branch_unit_tb.v`, 14 checks. Every branch taken and not taken, `blt` vs `bltu` on the same inputs, and the two unused codes.

5 State

5.1 regfile

File: `register.v` **Purpose:** one 32-bit register.

On the rising edge: `reset` clears it, otherwise `en` loads `d`. Reset wins if both are high.

Naming note: despite the name, `regfile` is a single register. The full register file is `reg32`.

Testbench: `regfile_tb.v`, 7 checks. Load, hold when `en = 0`, no change before the clock edge, and reset priority.

5.2 reg32

File: `32register.v` **Purpose:** the 32 general registers, two read ports and one write port.

- A `generate` loop makes 31 `regfile`s for x1–x31. Each one is enabled only when `we` is high and `rd` matches it.
- x0 is not a register at all, just the constant 0. Writes to it go nowhere.
- Reads are combinational 32-to-1 muxes. Writes happen on the clock edge.
- **No write-through:** if a register is written and read in the same cycle, the read shows the *old* value until the edge. This matters for the pipeline.

```
wire [31:0] q [31:0];        // output of each register
assign q[0] = 32'b0;         // x0 always reads as zero
...
regfile r (... .en(we && (rd == i)), .d(wd), .q(q[i]));
...
assign rd1 = q[rs1];
assign rd2 = q[rs2];
```

- `q` is an array of 32 wires, one per register. `assign q[0] = 32'b0` is the whole implementation of x0: slot 0 is tied to zero and has no register behind it.
- `.en(we && (rd == i))`: inside the `generate` loop, `i` is that register's number, so only the register whose number matches `rd` is enabled, and only when `we` is high.
- `q[rs1]`: indexing an array with a signal builds a 32-to-1 mux. `rs1` picks which of the 32 values comes out. Two of these give the two read ports.

Testbench: `reg32_tb.v`, 10 checks. Reset, x0 stays 0, `we = 0`, both ports at once, x1 and x31, and the same-cycle write/read case.

5.3 memory

File: `memory.v` **Purpose:** one memory shared by instruction fetch and loads/stores.

- 4096 words (16 KiB), loaded at startup from `INIT_FILE` with `$readmemh`.
- **Instruction port:** combinational, word-aligned (`instr_addr[13:2]`).
- **Loads:** combinational. The word is shifted so the wanted byte or halfword lands at the bottom, then sign- or zero-extended by `funct3`. Gives 0 when `mem_read = 0`.
- **Stores:** on the clock edge. `sb` and `sh` write only their own byte lanes, so the rest of the word is untouched.
- Both ports see the same array, so a store is immediately visible to instruction fetch.

```
wire [ADDR_BITS-1:0] data_index = mem_addr[ADDR_BITS+1:2];
wire [31:0] word    = mem[data_index];
wire [31:0] shifted = word >> {mem_addr[1:0], 3'b000};

assign instr = mem[instr_addr[ADDR_BITS+1:2]];
```

- `mem_addr[ADDR_BITS+1:2]`: addresses count bytes, but the array holds words. Dropping the low 2 bits divides by 4 and gives the word number. With `ADDR_BITS = 12` that's bits 13:2, so 4096 words; higher address bits are ignored.
- `mem_addr[1:0]` is which byte *inside* the word (0–3). {`mem_addr[1:0], 3'b000`} glues three zeros on the end, which multiplies it by 8: the byte offset turned into a bit offset (0, 8, 16 or 24).
- `word >> ...` shifts the wanted byte or halfword down to bit 0. For example, address `0x602` is byte 2, so the word is shifted right 16 and the byte at `0x602` ends up in `shifted[7:0]`. The load `case` then only ever looks at `shifted[7:0]` or `shifted[15:0]` and extends it.
- `instr` uses the same word indexing for the fetch port, so the low 2 bits of the PC are ignored.

funct3	load	store	
000	lb	sb	byte, sign-extended
001	lh	sh	halfword, sign-extended
010	lw	sw	word
100	lbu	–	byte, zero-extended
101	lhu	–	halfword, zero-extended

Testbench: `memory_tb.v`, 27 checks. Loaded with the test program: fetches at known addresses, every one of the 141 instructions, every load width on a word with both negative and positive bytes, each store size at each byte position, stores that must change nothing, and both ports seeing the same data.

6 Top level

6.1 core

File: `core.v` **Purpose:** the single-cycle datapath. Connects all the modules above.

Every connection in the core is an `assign`. They're grouped here by job, in the same order as `core.v`.

Instruction fields

```
wire [4:0] rs1    = instr[19:15];
wire [4:0] rs2    = instr[24:20];
wire [4:0] rd     = instr[11:7];
wire [2:0] funct3 = instr[14:12];
```

These just name slices of the instruction. The register numbers are in the same bit positions in every format, so they can be wired straight to `reg32` without waiting for the decoder.

PC and jump targets

```
assign instr_addr    = pc;
assign pc_plus4      = pc + 32'd4;
assign branch_target = pc + imm;
assign jalr_target   = {alu_result[31:1], 1'b0};
```

- `instr_addr = pc`: the PC is the fetch address.
- `pc_plus4` is the next instruction in order. It's also the return address that `jal` and `jalr` write to `rd`.
- `branch_target = pc + imm` is used by every branch and by `jal`. The B and J immediates already have bit $0 = 0$ and are in bytes, so no extra shift is needed.
- {`alu_result[31:1], 1'b0`}: the ALU computes `rs1 + imm` for `jalr`. The concatenation keeps bits 31 down to 1 and puts a 0 in bit 0, because the ISA says the `jalr` target always has bit 0 cleared.

ALU input muxes

```
assign alu_a = (alu_a_sel == 2'b01) ? pc
             : (alu_a_sel == 2'b10) ? 32'b0
             : rs1_data;
assign alu_b = (alu_b_sel == 1'b1) ? imm : rs2_data;
```

- `alu_a`: `01` picks the PC (for `auipc`), `10` picks zero (for `lui`, so the ALU computes $0 + imm$), and anything else picks `rs1`. The unused code `11` also lands on `rs1`.
- `alu_b`: the immediate for I/S/U-type instructions, `rs2` for R-type.

Write-back mux

```
assign wb_data = (wb_sel == 2'b00) ? alu_result
               : (wb_sel == 2'b01) ? mem_rdata
               : (wb_sel == 2'b10) ? pc_plus4
               : muldiv_result;
```

A 4-to-1 mux choosing what gets written to `rd`: the ALU result, the loaded value, the return address, or the multiply/divide result.

Next PC

```
assign pc_next = (jalr)                  ? jalr_target
               : (jal)                   ? branch_target
               : (branch && branch_cond) ? branch_target
               : pc_plus4;
```

- The chain is checked top to bottom: `jalr`, then `jal`, then a taken branch, otherwise the next instruction. The decoder never sets more than one of `jalr`, `jal`, `branch` at once, so the order doesn't change any result.
- `branch && branch_cond` needs both. `branch` (from the decoder) says "this instruction is a branch"; `branch_cond` (from `branch_unit`) says "the comparison is true". The branch unit runs on *every* instruction, and an `add` whose `funct3` is `000` looks like a `beq` to it, so `branch_cond` alone would make ordinary instructions jump.
- This value is loaded into `pc` on the next rising clock edge.

Memory connections

```
assign mem_addr   = alu_result;
assign mem_wdata  = rs2_data;
assign mem_funct3 = funct3;
```

Loads and stores compute their address (`rs1 + imm`) in the ALU, a store always writes `rs2`, and `funct3` goes straight to memory to pick the size (byte, half, word) and whether a load is signed.

6.2 top

File: `top.v` **Purpose:** connect `core` to `memory`. Parameters: `RESET_PC`, `ADDR_BITS`, `INIT_FILE`. Inputs are just `clk` and `rst`.

7 Verification

7.1 Approach

- **Self-checking.** Every testbench compares outputs to expected values and prints PASS or FAIL. No waveform reading needed to know if it worked.
- **`!==`, not `!=`.** With `!=`, an output stuck at `x` makes the comparison `x`, which the `if` treats as false, so the test passes. `!==` compares `x` literally.

- **Edge cases first:** zero, all ones, the most negative number, shift by 31, divide by zero.
- **Bottom-up:** each module is tested before anything built on it.
- **Tests are tested.** Each testbench was run against a module that was broken on purpose, to make sure it actually fails.

7.2 Results

testbench	module	checks
`addersub_tb`	addersub	8
`logics_tb`	logics	6
`shifter_tb`	shifter	11
`compare_tb`	compare	7
`alu_tb`	alu	14
`branch_unit_tb`	branch_unit	14
`immgen_tb`	immgen	18
`multdiv_tb`	multdiv	21
`regfile_tb`	regfile	7
`reg32_tb`	reg32	10
`decoder_tb`	decoder	31
`memory_tb`	memory	27
`top_tb`	whole CPU	48
		222

All testbenches passed with the final number of tests being 222.

7.3 Test program

`test_asm.S` is a 141-instruction program that uses every RV32IM instruction type and stores 47 known answers in memory starting at `0x400`. `top_tb.v` runs the CPU until the program writes its DONE marker (or 2000 cycles pass), then checks all 47 answers and that the PC is parked on the final `j end`. The single-cycle core finishes in 262 cycles.

Because the testbench waits for DONE instead of counting cycles, it works unchanged on a pipelined core, and since many instructions use the result of the one right before them, the same program will test forwarding and stalls. Section 8 goes through it part by part.

7.4 Building and running

`build_hex.sh` turns a `.S` file into a `.hex` for `$readmemh` and a `.lst` listing (needs `llvm`):

```
./build_hex.sh testbenches/test_asm
```

Run a testbench with Icarus Verilog:

```
iverilog -Wall -o top_tb top.v core.v memory.v decoder.v alu.v logics.v \
    shifter.v compare.v addersub.v immgen.v branch_unit.v multdiv.v \
    32register.v register.v testbenches/top_tb.v && vvp top_tb
```

`$readmemh` paths are relative to the folder `vvp` runs in.

8 Test program walkthrough

`test_asm.S` is the program `top_tb.v` runs on the whole CPU. It isn't useful software. It's a list of small problems, each with a known answer, chosen so that every RV32IM instruction type gets used at least once. This section goes through it part by part: the code, what each line does, and the value every check expects.

8.1 How the program reports results

Each test stores its answer in its own word of memory, called a *box*. Box n is at address `0x400` $+ 4n$. The very first instruction points `x8` at box 0, and every store after that is an offset from it:

```
    sub  x3, x1, x2             # 5 - 7, goes negative
    sw   x3, 8(x8)              # [2] FFFFFFFE
```

The comment `[2] FFFFFFFE` means "box 2 should hold `FFFFFFFE`". Most tests compute into `x7` and then store it.

address	what's there
`0x000`–`0x230`	the program (141 instructions)
`0x400`–`0x4B8`	boxes 0–46, the answers
`0x600`	a spare word for the load/store tests

When it's done, the program writes `D0E0000D` into box 46 and then jumps to itself forever. `top_tb.v` waits for that marker, then reads every box back through the hierarchy (`dut.u_memory.mem[256 + n]`, since memory is indexed by word and `0x400`/4 = 256) and compares each one with its expected value.

8.2 Overview

part	addresses	in program	executed	boxes
setup	`000`	1	1	–
math	`004`–`028`	10	10	0–3
big numbers	`02C`–`048`	8	8	4–6
logic	`04C`–`074`	11	11	7–11
shifts	`078`–`0A8`	13	13	12–17
compares	`0AC`–`0CC`	9	9	18–21
loads and stores	`0D0`–`118`	19	19	22–27
branches	`11C`–`150`	14	11	28
loop	`154`–`168`	6	33	29
function calls	`16C`–`18C`	9	11	30–32
mul / div	`190`–`1F4`	26	26	33–43
fibonacci	`1F8`–`218`	9	104	44
x0	`21C`–`220`	2	2	45
done	`224`–`230`	4	4	46
total		141	262	

"Executed" counts the instructions that actually run. The branches part runs fewer than it contains

(three `+100`s are skipped), and the two loops run far more. 262 instructions is exactly the 262 cycles the single-cycle core takes to reach DONE.

8.3 Setup

```
8  _start:
9      addi x8, x0, 0x400          # x8 = box 0
```

`x8 = 0x400`, the address of box 0. Nothing is stored here, but if this line is wrong every store lands in the wrong place and *every* check fails, so it's the first thing to look at when that happens.

The immediate `0x400` has bit 10 set, which is `instr[30]`. That's exactly the bit the decoder has to ignore for `addi`; if it didn't, this line would decode as a subtract.

8.4 Math (boxes 0–3)

```
11 # math
12     addi x1, x0, 5              # x1 = 5
13     addi x2, x0, 7              # x2 = 7
14     add  x3, x1, x2             # 5 + 7
15     sw   x3, 0(x8)              # [0] 0000000C
16     sub  x3, x3, x1             # 12 - 5
17     sw   x3, 4(x8)              # [1] 00000007
18     sub  x3, x1, x2             # 5 - 7, goes negative
19     sw   x3, 8(x8)              # [2] FFFFFFFE
20     addi x3, x3, -2046          # big negative imm
21     sw   x3, 12(x8)             # [3] FFFFF800
```

Lines 12–13 load the two operands: `x1 = 5`, `x2 = 7`. Every following line uses a value written by the line just before it, which is trivial for a single-cycle core but is the main hazard case for a pipeline.

box	expected	why
0	`0000000C`	$5 + 7 = 12$.
1	`00000007`	$12 - 5 = 7$. `x3` is read right after it was written.
2	`FFFFFFFE`	$5 - 7 = -2$. In two's complement -2 is $\neg 2 + 1 =$ `FFFFFFFD` $+ 1$.
3	`FFFFF800`	$-2 + (-2046) = -2048$. The immediate is stored as 12 bits (`0x802`) and must be sign-extended to 32.

8.5 Big numbers (boxes 4–6)

```
23 # big numbers
24     lui  x4, 0x12345            # top 20 bits
25     addi x4, x4, 0x678          # bottom 12 bits
26     sw   x4, 16(x8)             # [4] 12345678
27     lui  x5, 0x80000
28     addi x5, x5, -1             # biggest positive
29     sw   x5, 20(x8)             # [5] 7FFFFFFF
30 here:
31     auipc x6, 0                 # own address
32     sw   x6, 24(x8)             # [6] 00000044
```

An immediate is only 12 bits, so a full 32-bit constant takes two instructions: `lui` sets the top 20 bits (and zeros the bottom 12), then `addi` adds the bottom 12.

box	expected	why
4	`12345678`	`lui` gives `12345000`, `addi 0x678` fills in the bottom.
5	`7FFFFFFF`	`80000000` -1: the largest positive 32-bit number.
6	`00000044`	`auipc x6, 0` puts its own address in `x6`. There are 17 instructions before it, so it's at $17 \times 4 = \texttt{0x44}$.

8.6 Logic (boxes 7–11)

```
# logic
    xori x6, x4, -1             # not
    sw   x6, 28(x8)             # [7] EDCBA987
    andi x7, x4, 0xFF           # keep low byte
    ori  x7, x7, 0x700          # set some bits
    sw   x7, 32(x8)             # [8] 00000778
    xor  x7, x4, x6             # x ^ ~x
    sw   x7, 36(x8)             # [9] FFFFFFFF
    and  x7, x4, x6             # x & ~x
    sw   x7, 40(x8)             # [10] 00000000
    or   x7, x1, x2             # 5 | 7
    sw   x7, 44(x8)             # [11] 00000007
```

`x4` still holds `12345678` from the last part.

box	expected	why
7	`EDCBA987`	`xori` with `-1` (all ones) flips every bit. Each hex digit d becomes $15 - d$: 1 → `E`, 2 → `D`, ...
8	`00000778`	`andi 0xFF` keeps only the low byte, `78`. `ori 0x700` then sets bits to give `778`.
9	`FFFFFFFF`	A value xor its own opposite: every bit differs.
10	`00000000`	A value and its own opposite: no bit is 1 in both.
11	`00000007`	`0101 | 0111 = 0111`.

8.7 Shifts (boxes 12–17)

```
# shifts
    slli x7, x1, 4              # 5 << 4
    sw   x7, 48(x8)             # [12] 00000050
    srli x7, x6, 28             # zeros fill in
    sw   x7, 52(x8)             # [13] 0000000E
    srai x7, x6, 28             # sign fills in
    sw   x7, 56(x8)             # [14] FFFFFFFE
    sll  x7, x1, x2             # 5 << 7
    sw   x7, 60(x8)             # [15] 00000280
    addi x9, x0, -1             # x9 = all ones
    srl  x7, x9, x1             # all ones >> 5
    sw   x7, 64(x8)             # [16] 07FFFFFF
    sra  x7, x6, x1             # negative >>> 5
    sw   x7, 68(x8)             # [17] FF6E5D4C
```

`x6` is `EDCBA987` from box 7. It's negative (top bit 1), which is what makes the logical and arithmetic right shifts give different answers.

box	expected	why
12	`00000050`	$5 \ll 4 = 5 \times 16 = 80$.
13	`0000000E`	Shift right 28: only the top hex digit `E` is left, zeros fill in.
14	`FFFFFFFE`	Same shift, but copies of the sign bit fill in. `E` is `1110`, so the result is all ones ending in `1110` (-2). Compare with box 13.
15	`00000280`	$5 \ll 7 = 640$. The shift amount comes from a register this time.
16	`07FFFFFF`	`x9` is all ones. Shifting right 5 puts five zeros on top.
17	`FF6E5D4C`	A logical shift right 5 of `EDCBA987` gives `076E5D4C`; `sra` fills those top five bits with ones instead.

8.8 Compares (boxes 18–21)

```
# compares
    addi x10, x0, -2            # x10 = -2
    slt  x7, x10, x1            # -2 < 5
    sw   x7, 72(x8)             # [18] 00000001
    sltu x7, x10, x1            # unsigned, FFFFFFFE is huge
    sw   x7, 76(x8)             # [19] 00000000
    slti x7, x10, -1            # -2 < -1
    sw   x7, 80(x8)             # [20] 00000001
    sltiu x7, x1, -1            # 5 < FFFFFFFF
    sw   x7, 84(x8)             # [21] 00000001
```

The answer is 1 for yes, 0 for no. `x10 = -2` is `FFFFFFFE`: a small negative number when read as signed, and a huge number when read as unsigned.

box	expected	why
18	`00000001`	Signed: $-2 < 5$.
19	`00000000`	Unsigned: `FFFFFFFE` < 5 is false. Same bits as box 18, opposite answer.
20	`00000001`	$-2 < -1$.
21	`00000001`	The immediate -1 is sign-extended to `FFFFFFFF` *before* the unsigned compare, so this asks $5 <$ `FFFFFFFF`.

8.9 Loads and stores (boxes 22–27)

```
73 # loads and stores
74     addi x11, x0, 0x600         # x11 = spare word
75     lui  x12, 0x80818
76     addi x12, x12, -255         # x12 = 80817F01
77     sw   x12, 0(x11)            # bytes: 01 7F 81 80
78     lb   x7, 2(x11)             # byte 81, signed
79     sw   x7, 88(x8)             # [22] FFFFFF81
80     lbu  x7, 2(x11)             # byte 81, unsigned
81     sw   x7, 92(x8)             # [23] 00000081
82     lh   x7, 2(x11)             # half 8081, signed
83     sw   x7, 96(x8)             # [24] FFFF8081
84     lhu  x7, 2(x11)             # half 8081, unsigned
85     sw   x7, 100(x8)            # [25] 00008081
86     lw   x7, 0(x11)             # whole word
87     addi x7, x7, 1              # used right away
88     sw   x7, 104(x8)            # [26] 80817F02
89     sb   x1, 1(x11)             # change one byte
90     sh   x2, 2(x11)             # change top half
91     lw   x7, 0(x11)             # read it back
92     sw   x7, 108(x8)            # [27] 00070501
```

Lines 75–76 build `80817F01`. The obvious way, `lui 0x80817` then `addi 0xF01`, doesn't work: `0xF01` has bit 11 set, so `addi` would sign-extend it to −255. Instead the `lui` value is one higher (`0x80818`) and the `addi` subtracts `0xFF`.

RISC-V is little-endian, so after the store the lowest byte sits at the lowest address:

address	600	601	602	603
byte after line 77	01	7F	81	80
byte after line 89	01	05	81	80
byte after line 90	01	05	07	00

box	expected	why
22	FFFFFF81	`lb` at 602 reads 81. Its top bit is 1, so it's sign-extended with ones.
23	00000081	`lbu`, same byte, zero-extended.
24	FFFF8081	`lh` at 602 reads bytes 602–603 as the half 8081, sign-extended.
25	00008081	`lhu`, same half, zero-extended.
26	80817F02	`lw` gives 80817F01, and the very next line adds 1. On a pipeline this is a load-use hazard that needs a stall.
27	00070501	`sb` writes 05 into byte 601 only; `sh` writes 0007 into 602–603 only. Byte 600 is untouched.

8.10 Branches (box 28)

```
94  # branches
95  # taken ones skip a +100, not taken ones add 1, 2, 4
96      addi x13, x0, 0             # score = 0
97      beq  x1, x1, b1             # taken
98      addi x13, x13, 100          # skipped
99  b1: bne  x1, x1, b2             # not taken
100     addi x13, x13, 1
101 b2: blt  x10, x1, b3            # taken
102     addi x13, x13, 100          # skipped
103 b3: bge  x10, x1, b4            # not taken
104     addi x13, x13, 2
105 b4: bltu x10, x1, b5            # not taken
106     addi x13, x13, 4
107 b5: bgeu x10, x1, b6            # taken
108     addi x13, x13, 100          # skipped
109 b6: sw   x13, 112(x8)           # [28] 00000007
```

`x13` keeps a score. A branch that should be *taken* jumps over an `addi +100`, so a `+100` only runs if a branch that should jump doesn't. A branch that should *not* be taken falls into an `addi` of 1, 2 or 4, so those only run if it correctly falls through.

line	branch	compares	result	score
97	`beq x1, x1`	$5 = 5$	taken	0
99	`bne x1, x1`	$5 \neq 5$	not taken	1
101	`blt x10, x1`	$-2 < 5$	taken	1
103	`bge x10, x1`	$-2 \geq 5$	not taken	3
105	`bltu x10, x1`	`FFFFFFFE` < 5 unsigned	not taken	7
107	`bgeu x10, x1`	`FFFFFFFE` ≥ 5 unsigned	taken	7

box	expected	why
28	`00000007`	$1 + 2 + 4$. A wrong score also says which branch failed: each extra 100 is a taken branch that fell through, and a missing 1, 2 or 4 points at the `bne`, `bge` or `bltu` that jumped when it shouldn't.

8.11 Loop (box 29)

```
111 # loop: 10 + 9 + ... + 1
112     addi x14, x0, 0             # total
113     addi x15, x0, 10            # count
114 loop:
115     add  x14, x14, x15          # total += count
116     addi x15, x15, -1           # count--
117     bne  x15, x0, loop          # until 0
118     sw   x14, 116(x8)           # [29] 00000037 (55)
```

A count-down loop. The `bne` at line 117 jumps back 8 bytes to `loop` until `x15` reaches zero, so the body runs 10 times.

pass	1	2	3	4	5	6	7	8	9	10
`x15` added	10	9	8	7	6	5	4	3	2	1
`x14` after	10	19	27	34	40	45	49	52	54	55

box	expected	why
29	`00000037`	55. The backward branch is taken 9 times and falls through once.

8.12 Function calls (boxes 30–32)

```
120 # function calls
121     addi x10, x0, 21
122     jal  x1, double             # call
123     sw   x10, 120(x8)           # [30] 0000002A (42)
124     jal  x1, double             # call again
125 ret2:
126     sw   x10, 124(x8)           # [31] 00000054 (84)
127     sw   x1, 128(x8)            # [32] 0000017C (return address)
128     jal  x0, after_func         # skip over the function
129 double:
130     add  x10, x10, x10          # x10 * 2
131     jalr x0, 0(x1)              # return
132 after_func:
```

`double` adds `x10` to itself and returns. `jal x1, double` saves the address of the next instruction (PC + 4) in `x1` and jumps; `jalr x0, 0(x1)` jumps back to whatever `x1` holds. Writing the link to `x0` throws it away, so `jal x0, after_func` on line 128 is just a plain jump over the function.

step	instruction (address)	effect	`x10`
1	`jal` (170)	`x1 = 174`, go to `188`	21
2	`add` (188), `jalr` (18C)	double, go to `174`	42
3	`sw` (174)	box 30 = 42	42
4	`jal` (178)	`x1 = 17C`, go to `188`	42
5	`add`, `jalr`	double, go to `17C`	84
6	`sw`, `sw`, `jal x0` (17C–184)	boxes 31, 32, skip to `190`	84

box	expected	why
30	`0000002A`	$21 \times 2 = 42$: the call happened and came back.
31	`00000054`	$42 \times 2 = 84$: a second call works too.
32	`0000017C`	`x1` after the second `jal`, which was at `178`: `178` + 4 = `17C`, the address of `ret2`.

8.13 Multiply and divide (boxes 33–43)

```
# mul / div
    addi x16, x0, 6            # x16 = 6
    addi x17, x0, -7           # x17 = -7
    mul  x7, x16, x17          # 6 * -7
    sw   x7, 132(x8)           # [33] FFFFFFD6
    lui  x18, 0x80000          # most negative
    mulh x7, x18, x18          # top half, signed
    sw   x7, 136(x8)           # [34] 40000000
    mulhu x7, x9, x9           # top half, unsigned
    sw   x7, 140(x8)           # [35] FFFFFFFE
    mulhsu x7, x9, x9          # top half, signed * unsigned
    sw   x7, 144(x8)           # [36] FFFFFFFF
    addi x19, x0, -20          # x19 = -20
    div  x7, x19, x16          # -20 / 6, rounds toward 0
    sw   x7, 148(x8)           # [37] FFFFFFFD
    rem  x7, x19, x16          # -20 % 6
    sw   x7, 152(x8)           # [38] FFFFFFFE
    divu x7, x9, x16           # unsigned divide
    sw   x7, 156(x8)           # [39] 2AAAAAAA
    remu x7, x9, x16           # unsigned remainder
    sw   x7, 160(x8)           # [40] 00000003
    div  x7, x16, x0           # divide by 0 :(
    sw   x7, 164(x8)           # [41] FFFFFFFF
    rem  x7, x16, x0           # remainder by 0
    sw   x7, 168(x8)           # [42] 00000006
    div  x7, x18, x9           # overflow
    sw   x7, 172(x8)           # [43] 80000000
```

Inputs: `x16 = 6`, `x17 = -7`, `x18 = 80000000` (-2^{31}), `x19 = -20`, and `x9 = FFFFFFFF` from the shifts part (which is -1 signed, or $2^{32}-1$ unsigned). The `mulh` variants return the *top* half of the 64-bit product.

box	expected	why
33	`FFFFFFD6`	$6 \times -7 = -42$.
34	`40000000`	$(-2^{31})^2 = 2^{62} =$ `40000000_00000000`; top half.
35	`FFFFFFFE`	$(2^{32}-1)^2 =$ `FFFFFFFE_00000001`; top half.
36	`FFFFFFFF`	First operand signed (-1), second unsigned ($2^{32}-1$): $-(2^{32}-1) =$ `FFFFFFFF_00000001`; top half.
37	`FFFFFFFD`	$-20/6 = -3.33$, which rounds toward zero to -3 (not -4).
38	`FFFFFFFE`	$-20 - (6 \times -3) = -2$. The remainder takes the sign of the dividend.
39	`2AAAAAAA`	$4294967295/6 = 715827882$.
40	`00000003`	$4294967295 - 6 \times 715827882 = 3$.
41	`FFFFFFFF`	Divide by zero. RISC-V doesn't trap; it returns all ones.
42	`00000006`	Remainder by zero returns the dividend.
43	`80000000`	$-2^{31}/-1 = 2^{31}$ doesn't fit in 32 bits, so the result is defined as -2^{31}.

8.14 Fibonacci (box 44)

```
# fibonacci, 20 steps
    addi x20, x0, 0             # a = 0
    addi x21, x0, 1             # b = 1
    addi x22, x0, 20            # steps
fib:
    add  x23, x20, x21          # next = a + b
    addi x20, x21, 0            # a = b
    addi x21, x23, 0            # b = next
    addi x22, x22, -1           # steps--
    bne  x22, x0, fib           # until 0
    sw   x20, 176(x8)           # [44] 00001A6D (6765)
```

Starting from $a = 0$, $b = 1$, each pass computes $a + b$ and slides the pair forward. After k passes, a is the k-th Fibonacci number. Five instructions per pass, 20 passes, plus setup and the store: 104 executed.

pass	1	2	3	4	5	6	7	8	9	10
x20	1	1	2	3	5	8	13	21	34	55
pass	11	12	13	14	15	16	17	18	19	20
x20	89	144	233	377	610	987	1597	2584	4181	6765

box	expected	why
44	`00001A6D`	fib(20) = 6765. Each pass depends on the last two results, so this is a long chain of back-to-back dependencies.

8.15 x0 (box 45)

```
# x0 stays 0
    addi x0, x0, 123            # ignored
    sw   x0, 180(x8)            # [45] 00000000
```

box	expected	why
45	`00000000`	`x0` is hardwired to zero, so the `addi` into it does nothing.

8.16 Done (box 46)

```
# done
    lui  x7, 0xD0E00
    addi x7, x7, 0x0D           # "DONE"
    sw   x7, 184(x8)            # [46] D0E0000D
end:
    jal  x0, end                # wait here forever
```

box	expected	why
46	`D0E0000D`	Written last, so seeing it means every earlier box has been written.

`jal x0, end` jumps to its own address (offset 0), so the CPU stays there forever instead of running off into uninitialized memory. The testbench checks that the PC is `0x230`.

8.17 Final register values

Not checked by the testbench (it checks memory), but handy when reading a waveform. Registers not listed are 0.

reg	value	from	reg	value	from
x1	0000017C	return address	x13	00000007	branch score
x2	00000007	7	x14	00000037	loop total
x3	FFFFF800	box 3	x15	00000000	loop count
x4	12345678	box 4	x16	00000006	6
x5	7FFFFFFF	box 5	x17	FFFFFFF9	-7
x6	EDCBA987	box 7	x18	80000000	-2^{31}
x7	D0E0000D	DONE marker	x19	FFFFFFEC	-20
x8	00000400	box 0 address	x20	00001A6D	fib(20)
x9	FFFFFFFF	all ones	x21	00002AC2	fib(21)
x10	00000054	84	x22	00000000	steps left
x11	00000600	spare word	x23	00002AC2	last `next`
x12	80817F01	load/store word			

All expected values in this section were checked twice: worked out by hand, and by running the same `.hex` through a separate RISC-V instruction simulator.

9 Known limitations

None of these affect correct programs on the current design. Most are handled in Stage 3.

- **Loose decoding.** The decoder doesn't reject invalid encodings: `funct7` isn't fully checked, M is detected from `instr[25]` alone, and unknown opcodes do nothing instead of trapping.
- `ecall`, `ebreak`, `fence` are nops for now.
- **Misaligned loads and stores** aren't supported (the one failing riscv-test, `ma_data`).
- **Combinational memory reads.** Fine in simulation, but an FPGA block RAM needs a clocked read, so this will change before it runs on the board.
- **Combinational divider.** A full 32-bit divide in one cycle is a very long path and will limit the clock speed on hardware.

10 Next steps

1. 5-stage pipeline (IF, ID, EX, MEM, WB) with forwarding, stalls and flushes
2. CSRs, traps, timer, memory-mapped I/O.
3. Board peripherals: LEDs, 7-segment display, UART, VGA.